

Local DNS Server Report

Name: Rui Yuhan, Student ID: 12310520

Code Implementation

Task 1: Iterative Query

1.1 IP address detection

```
1 def get_local_ip():
2     try:
3         # Connect to server to determine outbound interface
4         test_sock=socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
5         test_sock.connect(('8.8.8.8', 80))
6         ip=test_sock.getsockname()[0]
7         test_sock.close()
8         return ip
9     except Exception:
10        return '0.0.0.0'
```

Explanation:

This function detects the local IP by connecting to an external server (8.8.8.8) and using getsockname() to get the outbound interface IP.

1.2.1 Server Initialization

```
1 class DNSServer:
2     def __init__(self, source_ip, source_port, ip='127.0.0.1', port=5533,
3 num_workers=20):
4         self.source_ip = source_ip
5         self.source_port = source_port
6         self.ip = ip
7         self.port = port
8         # Create UDP socket for DNS communication
9         self.socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
10
11        # Initialize thread-safe queues for multi-threading
12        self.request_queue = Queue(maxsize=1000)
13        self.response_queue = Queue(maxsize=1000)
14        self.stop_event = threading.Event()
15        self.num_workers = num_workers
16
17        self.cache_manager = CacheManager()
18
19        self.receiver_thread=None
20        self.sender_thread=None
21        self.workers=[]
```

```
21
22     # Configure socket for better performance
23     self.socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
24     self.socket.settimeout(1.0)
25
```

Explanation:

Initializes the DNS server with UDP socket, thread-safe queues for request/response handling, and cache manager for performance optimization.

1.2.2 Server Management

```
1  class DNSServer:
2  ...
3  def stop(self):
4      if self.stop_event.is_set():
5          return
6      self.stop_event.set()
7
8      try:
9          self.socket.close()
10     except Exception:
11         pass
12
13     try:
14         self.response_queue.put_nowait(((self.ip,self.port),b""))
15     except Exception:
16         pass
17
18     try:
19         self.cache_manager.save_to_file()
20     except Exception:
21         pass
22     print("[DNSServer] Stopped.")
23
24  def _receive_loop(self):
25      while not self.stop_event.is_set():
26          try:
27              # Receive DNS requests from clients
28              data, address = self.socket.recvfrom(4096)
29              self.request_queue.put((data, address))
30          except socket.timeout:
31              continue
32          except OSError:
33              break
34          except Exception:
35              continue
36
37  def _send_loop(self):
38      while not self.stop_event.is_set():
```

```

39         try:
40             # Get response from queue and send to client
41             address,payload=self.response_queue.get(timeout=0.5)
42             if not payload:
43                 continue
44             self.socket.sendto(payload, address)
45         except Empty:
46             continue
47         except OSError:
48             break
49         except Exception:
50             continue
51

```

Explanation:

- `stop()`: Gracefully shuts down the server by setting stop event, closing socket, and saving cache to disk
- `_receive_loop()`: Continuously listens for incoming DNS requests on the socket and puts them into request queue for worker threads to process
- `_send_loop()`: Processes responses from response queue and sends them back to clients using the same socket

1.3 DNS Query Processing

```

1  class DNSHandler(threading.Thread):
2      ...
3      def handle(self, message):
4          ...
5          # Check cache first for performance
6          cached = self.cache_manager.readCache(domain_name, qtype_str)
7          if cached:
8              cached.header.id = income_record.header.id
9              return cached
10
11         # Perform iterative DNS query if not in cache
12         rr_list = self.query(domain_name, QTYPE.get(income_record.q.qtype) or "A")
13
14         if rr_list:
15             # Create response with found records
16             response = ReplyGenerator.myReply(income_record, rr_list)
17             # Cache the response
18             self.cache_manager.writeCache(domain_name, qtype_str, response)
19             return response
20         else:
21             # Domain not found, cache NXDOMAIN response
22             nxdomain_response = ReplyGenerator.replyForNotFound(income_record)
23             self.cache_manager.writeCache(domain_name, qtype_str, nxdomain_response)
24             return nxdomain_response

```

Explanation:

Parses DNS requests, checks cache first, performs iterative queries if needed, writes successful responses and NXDOMAIN responses to cache, and generates appropriate responses with matching request IDs.

1.4 Query Root

```

1  def queryRoot(self, source_ip, source_port):
2      # Configure resolver with timeout settings
3      res=resolver.Resolver(configure=True)
4      res.lifetime=2
5      res.timeout=2
6
7      # Try each bootstrap DNS server
8      bootstrap=getattr(self, "BOOTSTRAP_DNS_SERVERS", ['223.5.5.5', '119.29.29.29',
9      '8.8.8.8', '1.1.1.1'])
10     for dns_ip in bootstrap:
11         try:
12             res.nameservers=[dns_ip]
13             # Query for root NS records
14             ans=res.resolve(dns_name.from_text('.'),
15                             rdatatype.NS,raise_on_no_answer=True)
16             ns_name=str(ans.rrset[0].target).rstrip('.')
17             # Resolve NS name to get IP address
18             a_ans=res.resolve(ns_name, rdatatype.A,raise_on_no_answer=True)
19             root_ip=a_ans.rrset[0].address
20             return (root_ip, ns_name)
21         except Exception:
22             continue
23     raise Exception("Cannot discover root server from bootstrap resolvers.")

```

Explanation:

Dynamically discovers root DNS server IP by querying bootstrap servers for root NS records, then resolving the NS name to get the actual IP address for iterative queries.

1.5 Iterative Query Implementation

```

1  def query(self, query_name, qtype):
2      # Start from root server
3      current_server_ip = self.root_server_cache
4      if not current_server_ip:
5          current_server_ip, _ = self.queryRoot(self.source_ip, self.source_port)
6
7      collected_rrs = []
8      qname = query_name
9
10     # Iterate through DNS hierarchy (max 20 hops)

```

```

11     for _ in range(20):
12         qtype_name = qtype if isinstance(qtype, str) else (QTYPE.get(qtype) or "A")
13         q = DNSRecord.question(qname, qtype_name)
14         # Set RD flag to 0 for iterative query
15         q.header.rd = 0
16
17         sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
18         sock.settimeout(2.0)
19         try:
20             # Send query to current server
21             sock.sendto(q.pack(), (current_server_ip, 53))
22             data, _ = sock.recvfrom(4096)
23         except Exception:
24             try:
25                 current_server_ip, _ = self.queryRoot(self.source_ip,
self.source_port)
26                 continue
27             except Exception:
28                 return None
29         finally:
30             sock.close()
31
32         resp = DNSRecord.parse(data)
33
34         # Check for NXDOMAIN
35         if resp.header.rcode == 3:
36             return None
37
38         answers = resp.rr
39         auth = resp.auth
40         addi = resp.ar
41
42         # If we got answers, handle CNAME chains
43         if answers:
44             cname_switched = False
45             for rr in answers:
46                 rtype = QTYPE.get(rr.rtype)
47                 if rtype == "CNAME":
48                     collected_rrs.append(rr)
49                     qname = str(rr.rdata.label).rstrip(".")
50                     cname_switched = True
51             if cname_switched:
52                 continue
53
54             final_rrs = collected_rrs + answers
55             return final_rrs
56
57         # Extract NS records from authority section
58         next_ip = None
59         ns_names = []
60         for rr in auth:
61             if QTYPE.get(rr.rtype) == "NS":

```

```

62         ns_names.append(str(rr.rdata.label).rstrip("."))
63
64     # Build glue record map from additional section
65     glue_map = {}
66     for rr in addi:
67         rtype = QTYPE.get(rr.rtype)
68         if rtype in ("A", "AAAA"):
69             glue_map[str(rr.rname).rstrip(".")] = rr
70
71     # Find next server IP from glue records
72     for ns in ns_names:
73         if ns in glue_map:
74             rr = glue_map[ns]
75             if QTYPE.get(rr.rtype) == "A":
76                 next_ip = str(rr.rdata)
77                 break
78
79     # If no glue record, resolve NS name separately
80     if not next_ip and ns_names:
81         res = resolver.Resolver(configure=True)
82         res.lifetime = 2.0
83         res.timeout = 2.0
84         res.nameservers = ['223.5.5.5', '119.29.29.29', '8.8.8.8', '1.1.1.1']
85         for ns in ns_names:
86             try:
87                 ans = res.resolve(ns, rdatatype.A, raise_on_no_answer=True)
88                 next_ip = ans[0].address
89                 break
90             except Exception:
91                 continue
92
93     if not next_ip:
94         try:
95             current_server_ip, _ = self.queryRoot(self.source_ip,
self.source_port)
96             continue
97         except Exception:
98             return None
99
100     current_server_ip = next_ip
101
102     return None

```

Explanation:

- Starts from root server and queries each level of DNS hierarchy until finding the authoritative server
- Handles CNAME redirects by following the chain and collecting all CNAME records
- Extracts NS records from authority section to find next-level servers
- Uses glue records from additional section to get server IPs without extra queries

- Falls back to separate NS resolution if no glue records are available
- Implements proper error handling for NXDOMAIN and timeout scenarios

Task 2: Caching

2.1 Load Cache from File

```
1 def _load_from_file(self):
2     try:
3         # Load cache data from disk file
4         with open(self.cache_file, 'rb') as f:
5             data=pickle.load(f)
6     except (FileNotFoundError, EOFError, pickle.UnpicklingError, OSError):
7         return OrderedDict()
8
9     # Filter out expired entries based on current time
10    now=time.time()
11    cleaned=OrderedDict()
12    for key,value in (data.items() if isinstance(data, dict) else []):
13        try:
14            record,expires=value
15            if now<float(expires):
16                cleaned[key]= (record,float(expires))
17        except Exception:
18            continue
19    return cleaned
```

Explanation:

Loads cache from disk file, filters out expired entries based on current time, and returns only valid cache entries.

2.2 Save Cache to File

```
1 def save_to_file(self):
2     with self.lock:
3         try:
4             with open(self.cache_file, 'wb') as f:
5                 pickle.dump(self.cache, f,protocol=pickle.HIGHEST_PROTOCOL)
6         except Exception:
7             pass
```

Explanation:

Saves current in-memory cache to disk file using pickle serialization with thread-safe locking.

2.3 Read Cache and TTL

```

1 def readCache(self, domain_name, qtype_str):
2     if not domain_name or not qtype_str:
3         return None
4
5     # Create cache key from domain and query type
6     key = f"{domain_name.lower()}:{qtype_str.upper()}"
7     now = time.time()
8     with self.lock:
9         item = self.cache.get(key)
10        if not item:
11            return None
12        record, expires = item
13        # Check if record is still valid (not expired)
14        if expires > now:
15            # Update LRU order
16            self.cache.move_to_end(key, last=True)
17            return record
18        try:
19            # Remove expired entry
20            del self.cache[key]
21        except KeyError:
22            pass
23        return None

```

Explanation:

Creates cache key by combining domain name and query type with colon separator, then checks if the cached record exists and is still valid by comparing current time with expiration timestamp. If valid, updates LRU order and returns the record; if expired, removes the entry from cache and returns None to trigger a new network query.

2.4 Write Cache and TTL

```

1 def writeCache(self, domain_name, qtype_str, response_record):
2     if not domain_name or not qtype_str or not response_record:
3         return
4
5     # Create cache key from domain and query type
6     key = f"{domain_name.lower()}:{qtype_str.upper()}"
7     now = time.time()
8
9     # Determine TTL based on response type
10    if response_record.header.rcode == 3: # NXDOMAIN
11        ttl = 60 # 60 seconds for NXDOMAIN
12    else:
13        ttl = 300 # 300 seconds for normal responses
14
15    expires = now + ttl
16
17    with self.lock:
18        # Enforce LRU eviction if cache is full

```



```

19         if len(self.cache) >= self.max_size and key not in self.cache:
20             self.cache.popitem(last=False) # Remove least recently used
21
22         # Store record with expiration time
23         self.cache[key] = (response_record, expires)

```

Explanation:

Determines TTL based on response type (NXDOMAIN gets 60s, normal responses get 300s), calculates expiration time by adding TTL to current time, implements LRU eviction when cache reaches maximum size, and stores the record with expiration timestamp using domain:querytype as the cache key.

Task 3: Optional Functionalities

3.1 Concurrent Request Handling

Explanation:

The concurrent request handling is implemented in the DNS server architecture from sections 1.2.1 and 1.2.2, which uses multi-threaded design with receiver/sender threads and worker pool to process multiple DNS requests simultaneously using producer-consumer pattern.

3.2 DNS Redirection

```

1  def replyForRedirect(income_record, redirect_ip, ttl=300):
2      # Create response header with same ID as request
3      header = DNSHeader(id=income_record.header.id, qr=1, ra=1)
4      response = DNSRecord(header, q=income_record.q)
5
6      # Create A record with redirect IP
7      a_record = RR(income_record.q.qname, QTYPE.A, rdata=A(redirect_ip), ttl=ttl)
8      response.add_answer(a_record)
9
10     return response
11
12     # In DNSHandler.handle():
13     if domain_name in self.redirect_map:
14         redirect_ip = self.redirect_map[domain_name]
15         response = ReplyGenerator.replyForRedirect(income_record, redirect_ip)
16         # Cache the redirect response
17         self.cache_manager.writeCache(domain_name, qtype_str, response)
18         return response

```

Explanation:

Creates custom DNS responses with forged A records pointing to redirect IPs, checks domain against redirect_map, caches the redirect response for better performance, and returns responses with specified TTL.

3.3 DNS Filtering

```

1  def replyForBlocked(income_record, reason="Blocked due to security policy"):
2      # Create response header with REFUSED status (RCODE=5)
3      header = DNSHeader(id=income_record.header.id, qr=1, rcode=5)
4      response = DNSRecord(header, q=income_record.q)
5
6      # Add TXT record with reason if provided
7      if reason:
8          txt_record = RR(income_record.q.qname, QTYPE.TXT, rdata=TXT(reason), ttl=300)
9          response.add_answer(txt_record)
10
11     return response
12
13 # In DNSHandler.handle():
14 if domain_name in self.blocklist:
15     response = ReplyGenerator.replyForBlocked(income_record)
16     # Cache the blocked response
17     self.cache_manager.writeCache(domain_name, qtype_str, response)
18     return response

```

Explanation:

Generates REFUSED responses (RCODE=5) for blocked domains, includes TXT records with blocking reason, caches the blocked response to avoid repeated checks, and checks domain against blocklist before normal resolution.

Results

Task 1: Iterative Query

=====

All 8 queries completed in 0.91 seconds.

=====

DNS Query Test Summary:

✓ SUCCESS (7 queries):

(0.01s) - www.google-analytics.com	-> 0.0.0.0
(0.01s) - www.google.com	-> 127.0.0.1
(0.19s) - www.tsinghua.edu.cn	-> 101.6.15.66
(0.47s) - www.sina.com.cn	-> 121.194.5.14
(0.49s) - www.baidu.com	-> 182.61.200.108
(0.84s) - www.bilibili.com	-> 8.134.32.222
(0.91s) - www.taobao.com	-> 222.192.186.122

⚠ NXDOMAIN (0 queries):
None

✗ TIMEOUT (0 queries):
None

? OTHER ERRORS (1 queries):
(0.01s) - ads.annoying-tracker.com -> REFUSED

=====

Figure 1. DNS Server Test Results

Test Results: Successfully tested the iterative query functionality of the local DNS server with 20 concurrent threads querying 7 different domains, verifying proper handling of A records and CNAME records resolution along with multi-threaded concurrent processing capabilities.

```
--- [Thread-00] Result for www.baidu.com (took 0.45s) ---
; <<>> DiG 9.10.6 <<>> @127.0.0.1 www.baidu.com a -p 5533
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29684
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.baidu.com.                IN      A

;; ANSWER SECTION:
www.baidu.com.                 1200    IN      CNAME   www.a.shifen.com.
www.a.shifen.com.             120     IN      A       183.2.172.17
www.a.shifen.com.             120     IN      A       183.2.172.177

;; Query time: 429 msec
;; SERVER: 127.0.0.1#5533(127.0.0.1)
;; WHEN: Sun Oct 26 15:06:39 CST 2025
;; MSG SIZE rcvd: 90
```

Figure 2. Baidu Domain Resolution Test

Baidu Test Results: Detailed testing of www.baidu.com domain resolution showing successful A record and CNAME chain resolution through the iterative DNS query process.

```
> Frame 42: Packet, 533 bytes on wire (4264 bits), 533 bytes captured (4264 bits) on interface \Device\NPF_{F6BE6784-0924-4346-A4B7-61A34A020A5A},
> Ethernet II, Src: IETF-VRRP-VRID_02 (00:00:5e:00:01:02), Dst: AzureWaveTec_b3:33:8d (e8:fb:1c:b3:33:8d)
> Internet Protocol Version 4, Src: 198.41.0.4, Dst: 10.25.237.165
> User Datagram Protocol, Src Port: 53, Dst Port: 62612
√ Domain Name System (response)
  Transaction ID: 0x19ed
  > Flags: 0x8000 Standard query response, No error
  Questions: 1
  Answer RRs: 0
  Authority RRs: 13
  Additional RRs: 14
  √ Queries
    > www.baidu.com: type A, class IN
  √ Authoritative nameservers
    > com: type NS, class IN, ns j.gtld-servers.net
    > com: type NS, class IN, ns f.gtld-servers.net
    > com: type NS, class IN, ns l.gtld-servers.net
    > com: type NS, class IN, ns c.gtld-servers.net
    > com: type NS, class IN, ns b.gtld-servers.net
    > com: type NS, class IN, ns e.gtld-servers.net
    > com: type NS, class IN, ns k.gtld-servers.net
    > com: type NS, class IN, ns h.gtld-servers.net
    > com: type NS, class IN, ns a.gtld-servers.net
    > com: type NS, class IN, ns g.gtld-servers.net
    > com: type NS, class IN, ns d.gtld-servers.net
    > com: type NS, class IN, ns i.gtld-servers.net
    > com: type NS, class IN, ns m.gtld-servers.net
  √ Additional records
    > a.gtld-servers.net: type A, class IN, addr 192.5.6.30
    > b.gtld-servers.net: type A, class IN, addr 192.33.14.30
    > c.gtld-servers.net: type A, class IN, addr 192.26.92.30
    > d.gtld-servers.net: type A, class IN, addr 192.31.80.30
    > e.gtld-servers.net: type A, class IN, addr 192.12.94.30
    > f.gtld-servers.net: type A, class IN, addr 192.35.51.30
    > g.gtld-servers.net: type A, class IN, addr 192.42.93.30
    > h.gtld-servers.net: type A, class IN, addr 192.54.112.30
    > i.gtld-servers.net: type A, class IN, addr 192.43.172.30
    > j.gtld-servers.net: type A, class IN, addr 192.48.79.30
    > k.gtld-servers.net: type A, class IN, addr 192.52.178.30
    > l.gtld-servers.net: type A, class IN, addr 192.41.162.30
    > m.gtld-servers.net: type A, class IN, addr 192.55.83.30
    > a.gtld-servers.net: type AAAA, class IN, addr 2001:503:a83e::2:30
  [Request In: 41]
  [Time: 33.980000 milliseconds]
```

Figure 3. Iterative Query Step 1 - Query Root Server

Step 1: Query the root DNS server to obtain the authoritative name servers for .com TLD, receiving NS records in the Authority section and corresponding IP addresses (glue records) in the Additional section, initiating the iterative resolution process.

```

> Frame 45: Packet, 387 bytes on wire (3096 bits), 387 bytes captured (3096 bits) on interface \Device\NPF_{F6BE6784-0924-4346-A4B7-61A34A020A5A},
> Ethernet II, Src: IETF-VRRP-VRID_02 (00:00:5e:00:01:02), Dst: AzureWaveTec_b3:33:8d (e8:fb:1c:b3:33:8d)
> Internet Protocol Version 4, Src: 192.48.79.30, Dst: 10.25.237.165
> User Datagram Protocol, Src Port: 53, Dst Port: 62613
> Domain Name System (response)
  Transaction ID: 0x8559
  > Flags: 0x8000 Standard query response, No error
  Questions: 1
  Answer RRs: 0
  Authority RRs: 5
  Additional RRs: 11
  > Queries
    > www.baidu.com: type A, class IN
  > Authoritative nameservers
    > baidu.com: type NS, class IN, ns ns2.baidu.com
    > baidu.com: type NS, class IN, ns ns3.baidu.com
    > baidu.com: type NS, class IN, ns ns4.baidu.com
    > baidu.com: type NS, class IN, ns ns1.baidu.com
    > baidu.com: type NS, class IN, ns ns7.baidu.com
  > Additional records
    > ns2.baidu.com: type A, class IN, addr 220.181.33.31
    > ns2.baidu.com: type AAAA, class IN, addr 240e:940:603:4:0:ff:b01b:589a
    > ns3.baidu.com: type A, class IN, addr 153.3.238.93
    > ns3.baidu.com: type A, class IN, addr 36.155.132.78
    > ns4.baidu.com: type A, class IN, addr 111.45.3.226
    > ns4.baidu.com: type A, class IN, addr 14.215.178.80
    > ns1.baidu.com: type A, class IN, addr 110.242.68.134
    > ns1.baidu.com: type AAAA, class IN, addr 240e:bf:b801:1002:0:ff:b024:26de
    > ns7.baidu.com: type A, class IN, addr 180.76.76.92
    > ns7.baidu.com: type AAAA, class IN, addr 240e:940:603:4:0:ff:b01b:589a
    > ns7.baidu.com: type AAAA, class IN, addr 240e:bf:b801:1002:0:ff:b024:26de
  [Request In: 43]
  [Time: 266.613000 milliseconds]

```

Figure 4. Iterative Query Step 2 - Query .com TLD Server

Step 2: Query the .com TLD server to obtain the authoritative name servers for baidu.com domain, receiving baidu.com NS records in the Authority section and their corresponding IP addresses in the Additional section.

```

> Frame 50: Packet, 103 bytes on wire (824 bits), 103 bytes captured (824 bits) on interface \Device\NPF_{F6BE6784-0924-4346-A4B7-61A34A020A5A}, i
> Ethernet II, Src: IETF-VRRP-VRID_02 (00:00:5e:00:01:02), Dst: AzureWaveTec_b3:33:8d (e8:fb:1c:b3:33:8d)
> Internet Protocol Version 4, Src: 36.155.132.78, Dst: 10.25.237.165
> User Datagram Protocol, Src Port: 53, Dst Port: 62614
> Domain Name System (response)
  Transaction ID: 0x99e9
  > Flags: 0x8400 Standard query response, No error
  Questions: 1
  Answer RRs: 1
  Authority RRs: 0
  Additional RRs: 0
  > Queries
    > www.baidu.com: type A, class IN
      Name: www.baidu.com
      [Name Length: 13]
      [Label Count: 3]
      Type: A (1) (Host Address)
      Class: IN (0x0001)
  > Answers
    > www.baidu.com: type CNAME, class IN, cname www.a.shifen.com
      Name: www.baidu.com
      Type: CNAME (5) (Canonical NAME for an alias)
      Class: IN (0x0001)
      Time to live: 1200 (20 minutes)
      Data length: 18
      CNAME: www.a.shifen.com
  [Request In: 46]
  [Time: 46.160000 milliseconds]

```

Figure 5. Iterative Query Step 3 - Query baidu.com Server, Receive CNAME

Step 3: Query baidu.com's authoritative server for www.baidu.com and receive CNAME record in the Answer section, showing www.baidu.com points to www.a.shifen.com, requiring further resolution.

```

> Frame 60: Packet, 366 bytes on wire (2928 bits), 366 bytes captured (2928 bits) on interface \Device\NPF_{F6BE6784-0924-4346-A4B
> Ethernet II, Src: IETF-VRRP-VRID_02 (00:00:5e:00:01:02), Dst: AzureWaveTec_b3:33:8d (e8:fb:1c:b3:33:8d)
> Internet Protocol Version 4, Src: 36.155.132.12, Dst: 10.25.237.165
> User Datagram Protocol, Src Port: 53, Dst Port: 62616
√ Domain Name System (response)
  Transaction ID: 0x37c4
  > Flags: 0x8400 Standard query response, No error
    Questions: 1
    Answer RRs: 2
    Authority RRs: 5
    Additional RRs: 9
  √ Queries
    > www.a.shifen.com: type A, class IN
  √ Answers
    > www.a.shifen.com: type A, class IN, addr 183.240.99.58
    > www.a.shifen.com: type A, class IN, addr 183.240.99.169
  √ Authoritative nameservers
    > a.shifen.com: type NS, class IN, ns ns4.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns5.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns1.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns2.a.shifen.com
    > a.shifen.com: type NS, class IN, ns ns3.a.shifen.com
  √ Additional records
    > ns1.a.shifen.com: type A, class IN, addr 110.242.68.42
    > ns2.a.shifen.com: type A, class IN, addr 220.181.33.32
    > ns3.a.shifen.com: type A, class IN, addr 36.155.132.12
    > ns3.a.shifen.com: type A, class IN, addr 153.3.238.162
    > ns4.a.shifen.com: type A, class IN, addr 14.215.177.229
    > ns4.a.shifen.com: type A, class IN, addr 111.20.4.28
    > ns5.a.shifen.com: type A, class IN, addr 180.76.76.95
    > ns5.a.shifen.com: type AAAA, class IN, addr 240e:bf:b801:1006:0:ff:b04f:346b
    > ns5.a.shifen.com: type AAAA, class IN, addr 240e:940:603:a:0:ff:b08d:239d
  [Request In: 58]
  [Time: 39.023000 milliseconds]

```

Figure 6. Iterative Query Step 4 - Resolve CNAME to Final IP

Step 4: Follow the CNAME chain and query for www.a.shifen.com to obtain the final A records with IP addresses in the Answer section, completing the entire iterative DNS resolution process and returning the result to the client.

Task 2: Caching

```

> dig @127.0.0.1 www.baidu.com a -p 5533

; <<>> DiG 9.10.6 <<>> @127.0.0.1 www.baidu.com a -p 5533
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4621
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;www.baidu.com.                IN      A

;; ANSWER SECTION:
www.baidu.com.                 1200    IN      CNAME   www.a.shifen.com.
.                               1200    IN      CNAME   www.a.shifen.com.
www.a.shifen.com.             120     IN      A       183.240.99.169
www.a.shifen.com.             120     IN      A       183.240.99.58

;; Query time: 281 msec
;; SERVER: 127.0.0.1#5533(127.0.0.1)
;; WHEN: Sun Oct 26 18:13:34 CST 2025
;; MSG SIZE rcvd: 90

```

Figure 7. DNS Cache Performance Test - Before Cache (Cache Miss)

Cache Miss Performance: First query to www.baidu.com showing cache miss scenario with query time of 281 msec, demonstrating the full iterative DNS resolution process including CNAME chain resolution.

```

> dig @127.0.0.1 www.baidu.com a -p 5533

; <<>> DiG 9.10.6 <<>> @127.0.0.1 www.baidu.com a -p 5533
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 42181
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 0
L: 0

;; QUESTION SECTION:
;www.baidu.com.                IN      A

;; ANSWER SECTION:
www.baidu.com.                 1200    IN      CNAME   www.a.shifen.com.
.                               1200    IN      CNAME   www.a.shifen.com.
www.a.shifen.com.             120     IN      A       183.240.99.169
www.a.shifen.com.             120     IN      A       183.240.99.58

;; Query time: 0 msec
;; SERVER: 127.0.0.1#5533(127.0.0.1)
;; WHEN: Sun Oct 26 18:13:37 CST 2025
;; MSG SIZE rcvd: 90

```

Figure 8. DNS Cache Performance Test - Cache Hit

Cache Hit Performance: Subsequent queries to www.baidu.com showing cache hit scenario with query time of 0 msec, demonstrating the dramatic performance improvement achieved through caching. The response time improvement from 281ms to 0ms represents an infinite speedup, proving the effectiveness of the caching mechanism.

Task 3: Optional Functionalities

```
DOMAINS_TO_TEST = [  
    # DNS Redirection Test Cases  
    "www.google.com",          # Google search - should redirect to 127.0.0.1  
    "google.com",             # Google root domain - should redirect to 127.0.0.1  
    "www.google-analytics.com", # Google Analytics - should redirect to 0.0.0.0  
    "doubleclick.net",        # DoubleClick ads - should redirect to 0.0.0.0  
    "friendly.name",          # Custom friendly domain - should redirect to 8.8.8.8  
]
```

Figure 9. DNS Redirection Configuration

Redirection Test Results: Successfully tested DNS redirection functionality with 5 different domains, verifying that Google services redirect to 127.0.0.1, ad tracking services redirect to 0.0.0.0, and friendly domains redirect to 8.8.8.8 as configured in the `redirect_map`.

```
✓ SUCCESS (5 queries):  
(0.01s) - friendly.name          -> 8.8.8.8  
(0.01s) - doubleclick.net        -> 0.0.0.0  
(0.01s) - google.com             -> 127.0.0.1  
(0.01s) - www.google-analytics.com -> 0.0.0.0  
(0.01s) - www.google.com         -> 127.0.0.1
```

Figure 10. DNS Redirection Test Results

Redirection Performance: All redirection queries completed successfully with fast response times (0.01-0.02s), demonstrating efficient redirection rule processing and custom DNS response generation.

```
DOMAINS_TO_TEST = [  
    # DNS Filtering Test Cases (should be blocked with REFUSED)  
    "malware-site.com",          # Malware domain - should be blocked  
    "phishing-attack.net",      # Phishing domain - should be blocked  
    "ads.annoying-tracker.com", # Ad tracker - should be blocked  
    "stats.unwanted-data-miner.org", # Data mining tracker - should be blocked  
    "distracting-social-media.com", # Social media distraction - should be blocked  
]
```

Figure 11. DNS Filtering Configuration

Filtering Test Results: Successfully tested DNS filtering functionality with 5 different malicious and unwanted domains, verifying that all blocked domains return REFUSED status (RCODE=5) as configured in the blocklist.

DNS Query Test Summary:

✓ SUCCESS (0 queries):
None

⚠ NXDOMAIN (0 queries):
None

✗ TIMEOUT (0 queries):
None

? OTHER ERRORS (5 queries):
(0.01s) – distracting-social-media.com -> REFUSED
(0.01s) – phishing-attack.net -> REFUSED
(0.01s) – malware-site.com -> REFUSED
(0.01s) – stats.unwanted-data-miner.org -> REFUSED
(0.01s) – ads.annoying-tracker.com -> REFUSED

Figure 12. DNS Filtering Test Results

Filtering Performance: All filtering queries completed with REFUSED responses in 0.01-0.02s, demonstrating effective domain blocking and security policy enforcement.

=====

All 25 queries completed in 0.87 seconds.

=====

 DNS Query Test Summary:

-  **SUCCESS (18 queries):**
- (0.01s) – friendly.name

→ 8.8.8.8
- (0.01s) – www.google.com

→ 127.0.0.1
- (0.01s) – doubleclick.net

→ 0.0.0.0
- (0.01s) – google.com

→ 127.0.0.1
- (0.01s) – www.google-analytics.com

→ 0.0.0.0
- (0.10s) – drive.google.com

→ 108.160.169.178
- (0.14s) – www.wikipedia.org

→ 199.16.158.182
- (0.26s) – www.fudan.edu.cn

→ 202.120.224.81
- (0.34s) – www.stackoverflow.com

→ 172.64.155.249
- (0.41s) – www.pku.edu.cn

→ 162.105.131.160
- (0.41s) – maps.google.com

→ 142.250.196.206
- (0.42s) – mail.google.com

→ 142.250.198.69
- (0.45s) – www.baidu.com

→ 183.240.99.169
- (0.55s) – www.sina.com.cn

→ 121.194.5.14
- (0.57s) – www.tsinghua.edu.cn

→ 101.6.15.66
- (0.69s) – www.taobao.com

→ 222.192.186.120
- (0.83s) – www.github.com

→ 20.205.243.166
- (0.87s) – www.bilibili.com

→ 8.134.50.24
-  **NXDOMAIN (2 queries):**
- (0.32s) – this-domain-does-not-exist.net → Domain Not Found

(0.32s) – nonexistent-domain-12345.com → Domain Not Found
-  **TIMEOUT (0 queries):**
- None
-  **OTHER ERRORS (5 queries):**
- (0.01s) – stats.unwanted-data-miner.org → REFUSED

(0.01s) – phishing-attack.net → REFUSED

(0.01s) – distracting-social-media.com → REFUSED

(0.01s) – ads.annoying-tracker.com → REFUSED

(0.01s) – malware-site.com → REFUSED

=====

Figure 13. DNS Server Comprehensive Performance Test

Performance Test Results: Comprehensive performance testing with 25 concurrent queries completed in 0.87 seconds, demonstrating excellent multi-threading capabilities and efficient request processing across normal resolution, redirection, and filtering operations. The server achieved sub-second response times, indicating robust performance under concurrent load.